# CUSTOMER REQUIREMENTS SPECIFICATION

## Projekt 6: API für den Digitalen Produktpass (DPP) im BaSyx Framework

### Customer


| Name | Mail |
|---|---|
| Markus Rentschler | rentschler@lehre.dhbw-stuttgart.de |
| Pawel Wojcik | pawel.wojcik@lehre.dhbw-stuttgart.de |


### History

| Version | Autor | Datum | Kommentar |
|---|---|---|---|
| 1.0 | Magnus Lörcher | 11.11.2025 | Abgabe Semester 3 |

## Table of contents



## 1 Introduction

This document defines and specifies the requirements for the implementation of a *REST-API* for the management of Digital Product Pass (DPP) acording to **DIN-EN 18222**. The implementation includes both a backend for the **REST-API** as well as a frontend for users to interact/manage DPP's with. The implementation will be built upon the BaSyx framework.
**DIN-EN 18222** describes an API for life cycle management as well as searching within the "Product Pass" which will be the foundation of the developed solution.
The project will be developed as an open source repo on GitHub and after the conclusion of the project merged in to the BaSyx repository.

This CRS will be the foundation for the folowing documents:
Software Requirements Specification (SRS), every other design, test and implementation document.

### Task specification

> The **DIN-EN 18222** "Digital Product Passport - Application Programming Interfaces (APIs) for the product passport lifecycle management and searchability" describes a REST-API which will be implemented in the BaSyx-Framework back- and frontend. The task specification is available here.

## 2 Scope

The main goal of this project is the implementation of a **REST-API** for the management of DPP's with in the BaSyx framework.

| Scope | Aim |
|---|---|
| API specification (**DIN-EN 18222**) | Create an OpenAPI-Specification for the **REST-API** described by **DIN-EN 18222** |
| Backend implementation | Implementation of the DPP **REST-API** backend using the **JAVA-Spring boot** framework. The backend will expose API endpoints for the frontend to provide the described DPP functionality. This includes merging existing API's of the BaSyx backend. |
| Frontend implementation | The frontend (DPP viewer) will provide a UI to the end users to interact with and manage the DPP using the DPP backend. The DPP viewer will be integrated in to the existing BaSyx frontend module. |
| Usability and workflow | Definition of a clear usability concept as well as a workflow for the DPP viewer based on existing solutions. |
| Infrastructure and Deployment | The project will be hosted on a publically available server for demonstration purposes. |
| Documentation | The enire project documentation will be hosted in the wiki of the GitHub repository. |

## 3 Business Processes - BP

### BP01: Usability concept and DPP-viewer

| | |
|---|---|
| ID | BP02 |
| Roles involved | Product manager, Developer, Tester |
| Result | The developed Usability concept has been converted in to a functional frontend for the **REST-API**. |
| Procedure | 1. Analyze the existing BaSyx solution.<br>2. Define a usability concept and workflow for the DPP-viewer.<br>3. Develop the frontedn application.<br>4. Create suitable DPP examples for showcasing.<br>5. Test the solution. |
| Checkpoints | • Creation of the usability concept.<br>• First successfull display of a DPP.<br>• Finish the DPP-viewer. |

### BP02: Hosting and Dokumentation

| | |
|---|---|
| ID | BP03 |
| Roles involved | Technical writer, Project Manager, Developer |
| Result | The entire project is fully documented and publically accessable. |
| Procedure | 1. Hosting the developed solution on a publicaly accessable demo Server<br>2. Finalize and collect the entire project documentation in the repositories wiki |
| Checkpoints | • The demo server is accessable publicaly<br>• The solution runs on the demo server<br>• The documentation is finalized |

## 4 Use cases

### UC01 create/register DPP

| | |
|---|---|
| ID | UC01 |
| Description | Create a new AAS instance or import an existing one and add additional information/submodles for the Dpp. |
| Roles involved | |

| | |
|---|---|
| Prerequisit | Existing BaSyx backend with AAS API |
| Procedure | Using the existing AAS API of BaSyx register or create a new AAS instance through the Dpp API. |
| Result | The user can create or import AAS instances and use the DPP for further management |

### UC02 read/search

| | |
|---|---|
| ID | UC02 |
| Description | Search through the DPP database and read data from it |
| Roles involved | |
| Prerequisit | Existing BaSy backend with AAS API |
| Procedure | Identify the DPP in question by either an ID(API endpoint: dpps/{dppId}) or by an AAS atribute. Show available information on the requested DPP in the Dpp viewer. |
| Result | The user can search through existing DPPs and show their information |

### UC03 Update DPPs

| | |
|---|---|
| ID | UC03 |
| Description | Update an existing DPPs status or submodles for lifecycle management |
| Roles involved | |
| Prerequisit | BaSyx API, Search DPPs |
| Procedure | Search for a DPP using the functionality described above. Edit the DPPs status to refelct changes in the lifecycle |
| Result | The user can manage the lifecycle of an existing DPP |

### UC04 display DPP information

| | |
|---|---|
| ID | UC04 |
| Description | The graphical user interface allows the user to interact with and manage their DPPs. Functioning as an abstraction layer of the backend API |
| Roles involved | |
| Prerequisit | DPP backend API is implemented (at least partialy) |

| | |
|---|---|
| Procedure | Implement a coherent and intuitive UI to provide an abstraction layer for the DPP backend. All functionality of the backend is covered in the frontend. |
| Result | The user can use the DPP viwer to manage their DPPs |

### UC05 Exporting DPPs

| | |
|---|---|
| ID | UCxx |
| Description | Export existing DPPs as JSON/AASX |
| Roles involved | |
| Prerequisit | BaSyx API, Search for DPPs |
| Procedure | Find the requested DPP in the existing ones. Give an option to export the DPP for sharing or future importing |
| Result | The user can export an existing DPP |

## 5 Functional Requirements

### FR01 OpenAPI-Specification

| | |
|---|---|
| ID | FR01 |
| Description | The REST-API for the Dpp is described in an OPEN-API-Specification based on the **DIN-EN 18222** |

### FR02 Norm conformity

| | |
|---|---|
| ID | FR02 |
| Description | The REST-API has to implement all endpoints and operations described in **DIN-EN 18222** |

> After the meeting with Herr Rentschler on (14.11.2025) this requirement is not needed anymore. For example the version management of the DPPs is not needed anymore.

### FR03 DPP search

| | |
|---|---|
| ID | FR03 |
| Description | The existing DPPs are searchable by ID or other DPP parameter. |

### FR04 Creation of Dpp examples

| | |
|---|---|
| ID | FR04 |

| | |
|---|---|
| Description | To demonstrate the capabilities of the Dpp viewer/backend a few example Dpps are created / chosen. |

### FR05 DPP export as JSON

| | |
|---|---|
| ID | FR05 |
| Description | see UC05 |

### FR06 Dpp-viewer/frontend

| | |
|---|---|
| ID | FR06 |
| Description | A web based frontend where users can use the REST-API backend for their Dpp lifecycle management |

## 6 Nonfunctional Requirements

### NFR01 Usability concept and workflow definition

| | |
|---|---|
| ID | NFR01 |
| Description | A usability concept and workflows for all possible user interactions in the Dpp-viewer are defined to enshure optimal user guidance |

### NFR02 Documentation creation

| | |
|---|---|
| ID | NFR02 |
| Description | The project documentation allows future users or developers to continue the project. Every importand development step is documented. A user documentation is created to allow users to setup the DPP lifecycle management |

### NFR03 Performance

| | |
|---|---|
| ID | NFR03 |
| Description | The DPP API calls are optimized for run time and server load. The p95 respons time for 100 simultanius read requests should <300ms. The p95 respons time for an indexed search with a hit before 10k entries should be <800ms |

### NFR04 Deployability

| | |
|---|---|
| ID | NFR04 |
| Description | There exists a containerized version of the project for easy deployment using docer |

### NFR05 Hosting on Demoserver

| | |
|---|---|
| ID | NFR05 |
| Description | The project is hosted on a publically accessible demo server for presentation, testing and development. |

### NFR06 Code quality

| | |
|---|---|
| ID | NFR06 |
| Description | The development process is documented and reviewd regularly to enshure adherence to best practices |

## 7 Build & Deployment specifications

| **Environment** | **Requirement** | **Tooling** |
|---|---|---|
| Lokal (Dev) | Isolated development environment for the BaSyx-Stack. | Docker-Compose File |
| Demo-Server | Publically hosted instance of the project | Docker-Compose File |
| Konfiguration | Easily changable configuration parameters (Database link, Password, etc.) | .env File |

## 8 References

| **NR.** | **Referenz** | **Titel** | **Version** | **Link** |
|---|---|---|---|---|
| 1 | DIN EN 18222 | Digital Product Passport - Application Programming Interfaces (APIs) for the product passport lifecycle management and searchability | 2025 | Link |
| 2 | IDTA-02035-1 | Digital Battery Passport - Part 1 | 2025 | Link |
| 3 | IDTA-02035-2 | Digital Battery Passport - Part 2 | 2025 | Link |
| 4 | IDTA-02035-3 | Digital Battery Passport - Part 3 | 2025 | Link |
| 5 | IDTA-02035-4 | Digital Battery Passport - Part 4 | 2025 | Link |
| 6 | IDTA-02035-5 | Digital Battery Passport - Part 5 | 2025 | Link |
| 7 | IDTA-02035-6 | Digital Battery Passport - Part 6 | 2025 | Link |
| 8 | IDTA-02035-7 | Digital Battery Passport - Part 7 | 2025 | Link |
| 9 | HARTING AAS | HARTING Han 24B Assembly ZSN1 - AAS | - | Link |
| 10 | HARTING DPP | HARTING Han 24B Assembly ZSN1 - AAS displayed in DPP structure | - | Link |

| NR. | Referenz | Titel | Version | Link |
| --- | --- | --- | --- | --- |
| 11 | SRS | Software Requirement Specification | 2025 | Link |
| 12 | Aufgabenstellung | Team 6: BaSyx API Aufgabenstellung | 2025 | Link |